Disciplina: Estrutura de Dados

PROJETO BUBBLESORT

ORDENAÇÃO DO VALOR DE MERCADO DE 1.000.000 DE EMPRESAS
ESTRUTURA DE DADOS S.A.

INTEGRANTES DO GRUPO:

JARLISSON BRITO SANTOS

BRUNNO PASSOS

LUIZ GUSTAVO

Linguagem: Java (JDK 21)
Novembro de 2026

1. Objetivo

O programa lê o arquivo `Empresas_BubbleSort.txt`, que tem 1.000.000 de registros no formato `Nome Fantasia | Inscrição Estadual | Valor de Mercado`. Ele ordena os registros pelo valor de mercado, **do maior para o menor**, usando o algoritmo **Bubble Sort**, e grava o resultado em `Empresas_Ordenadas.txt`, no mesmo formato da entrada.

2. Organização do código

Classe	Responsabilidade
Empresa	Representa um registro. Guarda o nome, a inscrição estadual, o valor em texto (para gravar o arquivo igual à entrada) e o valor em **centavos** (`long`), que é usado nas comparações.
BubbleSort	Implementa o algoritmo de ordenação decrescente e conta passadas, comparações e trocas.
ArquivoEmpresas	Lê o TXT com `BufferedReader`, ignorando linhas vazias ou inválidas, e grava o resultado com `BufferedWriter`.
Main	Executa as etapas na ordem (ler, ordenar e gravar), mede o tempo de cada uma e mostra as maiores e menores empresas.
TesteBubbleSort	Testes automáticos que verificam se o sistema funciona.

3. Como o Bubble Sort funciona

O Bubble Sort percorre o vetor comparando cada elemento com o vizinho da direita. Na ordem decrescente, se o elemento da esquerda for **menor** que o da direita, os dois trocam de lugar. Ao fim de cada passada, o menor valor que ainda estava fora do lugar terá "afundado" até o final do vetor, como uma bolha no sentido contrário. As passadas se repetem até não haver mais trocas.

Exemplo com os valores 5.000.000 (A), 3.000.000 (B) e 7.000.000 (C):

```
Início:      [A 5.000.000] [B 3.000.000] [C 7.000.000]
Passada 1:   compara A e B: 5M < 3M? não, mantém
             compara B e C: 3M < 7M? sim, troca  -> [A] [C] [B]   (B, o menor, foi para o fim)
Passada 2:   compara A e C: 5M < 7M? sim, troca  -> [C] [A] [B]
Passada 3:   nenhuma troca, o vetor está ordenado: C (7M), A (5M), B (3M)
```

Otimizações usadas (o algoritmo continua sendo Bubble Sort)

1. **Parada antecipada:** se uma passada não faz nenhuma troca, o vetor já está ordenado e o algoritmo termina. Por isso o melhor caso é O(n).
2. **Posição da última troca:** tudo o que vem depois da última troca de uma passada já está no lugar certo. Assim, a próxima passada só vai até essa posição, o que evita comparações inúteis.
3. **Vetor auxiliar de `long`:** os valores em centavos ficam num vetor `long[]` que é trocado junto com o vetor de empresas. As comparações leem memória contígua em vez de acessar um objeto por vez, o que deixa o programa bem mais rápido com 1 milhão de registros.
4. **Valores em centavos:** comparar `long` evita os erros de arredondamento de `double` (por exemplo, 0,1 + 0,2 ≠ 0,3).

Estabilidade

Os elementos só são trocados quando um valor é *estritamente* menor que o outro. Por isso, empresas com o mesmo valor de mercado continuam na ordem em que aparecem no arquivo, ou seja, o algoritmo é **estável**.

4. Complexidade

Caso	Comparações	Complexidade
Melhor (já ordenado)	$n - 1$	O(n)
Médio (dados aleatórios)	$\approx n^2/2$	$O(n^2)$
Pior (ordem inversa)	$n(n-1)/2$	$O(n^2)$

O algoritmo usa memória extra O(n), por causa do vetor auxiliar de `long`. Com n = 1.000.000, são feitas cerca de **500 bilhões de comparações**. Por isso o Bubble Sort demora muito em grandes volumes. Um algoritmo O(n log n), como Merge Sort ou Quick Sort, faria cerca de 20 milhões de comparações.

5. Testes realizados

Os testes ficam na classe `TesteBubbleSort`. Para executá-los: `java -cp out TesteBubbleSort Empresas_BubbleSort.txt`

Teste	O que verifica
Exemplo do enunciado	A, B e C devem sair na ordem C, A, B
Vazio e 1 elemento	Os casos extremos não causam erro
Já ordenado	A parada antecipada funciona: 1 passada e 0 trocas
Ordem inversa	No pior caso, o número de trocas é exatamente n(n−1)/2
Valores iguais	Empates mantêm a ordem original (estabilidade)
Conversão para centavos	"652069.19" vira 65206919, entre outros casos
20.000 aleatórios	O resultado é idêntico ao do `Arrays.sort` do Java
Leitura e gravação	Linhas vazias e inválidas são ignoradas e o formato de saída fica correto
Amostra do arquivo real	Os 50.000 primeiros registros reais são ordenados corretamente

Saída dos testes

```
=== Testes do Bubble Sort ===
[OK]     Exemplo do enunciado -> C, A, B
[OK]     Vetor vazio e vetor com 1 elemento não quebram
[OK]     Vetor já ordenado: 1 passada e 0 trocas (melhor caso O(n))
[OK]     Ordem inversa (pior caso): n(n-1)/2 trocas = 499500
[OK]     Valores iguais mantêm a ordem original (algoritmo estável)
[OK]     Conversão de valores para centavos
[OK]     20.000 registros aleatórios: resultado idêntico ao Arrays.sort do Java
Aviso: linha 4 ignorada (Linha fora do formato esperado: linha invalida)
[OK]     Leitura ignora linhas vazias/inválidas e gravação mantém o formato
[OK]     Primeiros 50.000 registros do arquivo real ordenados corretamente (5.75 s)

Resultado: 9 OK, 0 FALHA(S)
```

Tempo de execução por quantidade de registros

Registros	Comparações	Trocas	Tempo de ordenação
10.000	49.964.098	25.044.468	0,21 s
50.000	1.249.848.657	627.826.002	6,00 s
100.000	4.999.208.836	2.508.577.389	22,13 s
200.000	19.999.249.012	10.008.384.923	183,23 s
1.000.000	499.996.540.252	250.159.742.115	**75,23 min**

Quando a quantidade de registros dobra, o tempo cresce pelo menos quatro vezes, o que confirma o comportamento $O(n^2)$. A partir de cerca de 100 mil registros, o crescimento fica ainda maior, porque os vetores passam a não caber na memória cache do processador.

Execução completa (1.000.000 de registros)

```
=== Estrutura de Dados S.A. - Ordenação por Valor de Mercado (Bubble Sort) ===
Entrada: Empresas_BubbleSort.txt
Registros lidos: 1,000,000 (0.75 s)
Ordenando com Bubble Sort (maior -> menor)... isso pode demorar.
Ordenação concluída em 4514.04 s (75.23 min)
Passadas: 999,550 | Comparações: 499,996,540,252 | Trocas: 250,159,742,115
Arquivo gravado: Empresas_Ordenadas.txt (0.80 s)

Top 5 maiores valores:
  1. Empresa 96433 | 95438272-3 | 9999993.82
  2. Empresa 565378 | 79944601-5 | 9999979.28
  3. Empresa 156492 | 97364927-8 | 9999971.69
  4. Empresa 969173 | 18441995-5 | 9999941.95
  5. Empresa 133248 | 58445063-9 | 9999940.81
Menores valores:
  999996. Empresa 880221 | 18077550-2 | 100130.37
  999997. Empresa 332551 | 35742857-8 | 100115.79
  999998. Empresa 379491 | 45469411-6 | 100113.45
  999999. Empresa 375728 | 26893286-2 | 100103.58
  1000000. Empresa 339426 | 76878469-9 | 100012.64

Tempo total: 4515.59 s
```

6. Como compilar e executar

```
javac -encoding UTF-8 -d out src/*.java
java -cp out TesteBubbleSort Empresas_BubbleSort.txt                    (testes)
java -Xmx2g -cp out Main Empresas_BubbleSort.txt Empresas_Ordenadas.txt (arquivo completo)
java -cp out Main Empresas_BubbleSort.txt Saida_10mil.txt 10000         (só os 10 mil primeiros)
```

7. Código-fonte

Empresa.java

```
// PROJETO BUBBLESORT - ESTRUTURA DE DADOS S.A.
// Integrantes: JARLISSON BRITO SANTOS, BRUNNO PASSOS E LUIZ GUSTAVO

/**
 * Representa um registro do arquivo: Nome Fantasia | Inscrição Estadual | Valor de Mercado.
 * O valor é guardado em centavos (long) para comparar sem erros de arredondamento
 * de ponto flutuante, e o texto original é mantido para gravar o arquivo de saída igual à entrada.
 */
public class Empresa {
    private final String nomeFantasia;
    private final String inscricaoEstadual;
    private final String valorTexto;
    private final long valorCentavos;

    public Empresa(String nomeFantasia, String inscricaoEstadual, String valorTexto) {
        this.nomeFantasia = nomeFantasia;
        this.inscricaoEstadual = inscricaoEstadual;
        this.valorTexto = valorTexto;
        this.valorCentavos = converterParaCentavos(valorTexto);
    }

    /** Converte "652069.19" em 65206919 centavos (aceita também vírgula ou sem casas decimais). */
    static long converterParaCentavos(String texto) {
        String t = texto.trim().replace(',', '.');
        int ponto = t.indexOf('.');
        if (ponto < 0) return Long.parseLong(t) * 100;
        long inteiro = ponto == 0 ? 0 : Long.parseLong(t.substring(0, ponto));
        String dec = (t.substring(ponto + 1) + "00").substring(0, 2);
        return inteiro * 100 + Long.parseLong(dec);
    }

    /** Monta uma linha a partir do formato "Nome | IE | Valor". */
    public static Empresa deLinha(String linha) {
        String[] partes = linha.split("\\|");
        if (partes.length != 3) {
            throw new IllegalArgumentException("Linha fora do formato esperado: " + linha);
        }
        return new Empresa(partes[0].trim(), partes[1].trim(), partes[2].trim());
    }

    public String paraLinha() {
        return nomeFantasia + " | " + inscricaoEstadual + " | " + valorTexto;
    }

    public String getNomeFantasia() { return nomeFantasia; }
    public String getInscricaoEstadual() { return inscricaoEstadual; }
    public String getValorTexto() { return valorTexto; }
    public long getValorCentavos() { return valorCentavos; }

    @Override
    public String toString() { return paraLinha(); }
}
```

BubbleSort.java

```
// PROJETO BUBBLESORT - ESTRUTURA DE DADOS S.A.
// Integrantes: JARLISSON BRITO SANTOS, BRUNNO PASSOS E LUIZ GUSTAVO

/**
 * Implementação do Bubble Sort para ordenar empresas pelo valor de mercado,
 * do MAIOR para o MENOR.
 *
 * Ideia do algoritmo: percorre o vetor comparando vizinhos (j e j+1). Se o da
 * esquerda for MENOR que o da direita, eles trocam de lugar. A cada passada o
 * menor valor ainda fora do lugar "afunda" até o fim do vetor.
 *
 * Otimizações (continua sendo Bubble Sort):
 *  1. Se uma passada inteira não fizer nenhuma troca, o vetor já está ordenado e paramos.
 *  2. Guardamos a posição da última troca: tudo depois dela já está no lugar certo,
 *     então a próxima passada só vai até ali.
 *  3. As comparações são feitas num vetor de long (valores em centavos) que anda junto
 *     com o vetor de empresas. Isso evita acessar cada objeto na memória a cada comparação
 *     e deixa o programa bem mais rápido com 1.000.000 de registros.
 */
public class BubbleSort {
```

```
    /** Contadores para mostrar o trabalho feito pelo algoritmo. */
    private long comparacoes;
    private long trocas;
    private long passadas;

    public void ordenarDecrescente(Empresa[] empresas) {
        comparacoes = 0;
        trocas = 0;
        passadas = 0;
        int n = empresas.length;
        if (n < 2) return;

        long[] valores = new long[n];
        for (int i = 0; i < n; i++) {
            valores[i] = empresas[i].getValorCentavos();
        }

        int limite = n - 1;          // última posição que ainda precisa ser comparada
        while (limite > 0) {
            int ultimaTroca = 0;
            passadas++;
            for (int j = 0; j < limite; j++) {
                if (valores[j] < valores[j + 1]) {   // ordem decrescente: o maior vai para a esquerda
                    long tmpValor = valores[j];
                    valores[j] = valores[j + 1];
                    valores[j + 1] = tmpValor;

                    Empresa tmpEmpresa = empresas[j];
                    empresas[j] = empresas[j + 1];
                    empresas[j + 1] = tmpEmpresa;

                    ultimaTroca = j;
                    trocas++;
                }
            }
            comparacoes += limite;
            limite = ultimaTroca;    // se não houve troca, ultimaTroca = 0 e o laço termina
        }
    }

    public long getComparacoes() { return comparacoes; }
    public long getTrocas() { return trocas; }
    public long getPassadas() { return passadas; }
}
```

ArquivoEmpresas.java

```
// PROJETO BUBBLESORT - ESTRUTURA DE DADOS S.A.
// Integrantes: JARLISSON BRITO SANTOS, BRUNNO PASSOS E LUIZ GUSTAVO

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

/** Leitura e gravação dos arquivos TXT no formato "Nome | IE | Valor". */
public class ArquivoEmpresas {

    /**
     * Lê o arquivo de entrada. Linhas em branco são ignoradas.
     * @param limite quantidade máxima de registros a ler (0 = todos)
     */
    public static Empresa[] ler(Path arquivo, int limite) throws IOException {
        List<Empresa> lista = new ArrayList<>(limite > 0 ? limite : 1_000_000);
        try (BufferedReader leitor = Files.newBufferedReader(arquivo, StandardCharsets.UTF_8)) {
            String linha;
            int numeroLinha = 0;
            while ((linha = leitor.readLine()) != null) {
                numeroLinha++;
                if (linha.isBlank()) continue;
                try {
                    lista.add(Empresa.deLinha(linha));
                } catch (IllegalArgumentException e) {
                    System.err.println("Aviso: linha " + numeroLinha + " ignorada (" + e.getMessage() + ")");
                }
                if (limite > 0 && lista.size() >= limite) break;
            }
        }
        return lista.toArray(new Empresa[0]);
    }

    /** Grava os registros, um por linha, no mesmo formato da entrada. */
    public static void gravar(Path arquivo, Empresa[] empresas) throws IOException {
        try (BufferedWriter escritor = Files.newBufferedWriter(arquivo, StandardCharsets.UTF_8)) {
            for (Empresa e : empresas) {
                escritor.write(e.paraLinha());
                escritor.newLine();
            }
        }
```

```
    }
}
```

Main.java

```
// PROJETO BUBBLESORT - ESTRUTURA DE DADOS S.A.
// Integrantes: JARLISSON BRITO SANTOS, BRUNNO PASSOS E LUIZ GUSTAVO

import java.nio.file.Path;

/**
 * Programa principal.
 * Uso: java Main [arquivoEntrada] [arquivoSaida] [limiteDeRegistros]
 * Padrões: Empresas_BubbleSort.txt  Empresas_Ordenadas.txt  0 (todos)
 */
public class Main {
    public static void main(String[] args) throws Exception {
        Path entrada = Path.of(args.length > 0 ? args[0] : "Empresas_BubbleSort.txt");
        Path saida   = Path.of(args.length > 1 ? args[1] : "Empresas_Ordenadas.txt");
        int limite   = args.length > 2 ? Integer.parseInt(args[2]) : 0;

        System.out.println("=== Estrutura de Dados S.A. - Ordenação por Valor de Mercado (Bubble Sort) ===");
        System.out.println("Entrada: " + entrada.toAbsolutePath());

        long t0 = System.nanoTime();
        Empresa[] empresas = ArquivoEmpresas.ler(entrada, limite);
        long t1 = System.nanoTime();
        System.out.printf("Registros lidos: %,d (%.2f s)%n", empresas.length, (t1 - t0) / 1e9);

        System.out.println("Ordenando com Bubble Sort (maior -> menor)... isso pode demorar.");
        BubbleSort sorter = new BubbleSort();
        sorter.ordenarDecrescente(empresas);
        long t2 = System.nanoTime();
        System.out.printf("Ordenação concluída em %.2f s (%.2f min)%n", (t2 - t1) / 1e9, (t2 - t1) / 60e9);
        System.out.printf("Passadas: %,d | Comparações: %,d | Trocas: %,d%n",
                sorter.getPassadas(), sorter.getComparacoes(), sorter.getTrocas());

        ArquivoEmpresas.gravar(saida, empresas);
        long t3 = System.nanoTime();
        System.out.printf("Arquivo gravado: %s (%.2f s)%n", saida.toAbsolutePath(), (t3 - t2) / 1e9);

        int mostrar = Math.min(5, empresas.length);
        System.out.println("\nTop " + mostrar + " maiores valores:");
        for (int i = 0; i < mostrar; i++) System.out.println("  " + (i + 1) + ". " + empresas[i]);
        System.out.println("Menores valores:");
        for (int i = Math.max(0, empresas.length - mostrar); i < empresas.length; i++)
            System.out.println("  " + (i + 1) + ". " + empresas[i]);
        System.out.printf("%nTempo total: %.2f s%n", (t3 - t0) / 1e9);
    }
}
```

TesteBubbleSort.java

```
// PROJETO BUBBLESORT - ESTRUTURA DE DADOS S.A.
// Integrantes: JARLISSON BRITO SANTOS, BRUNNO PASSOS E LUIZ GUSTAVO

import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Random;

/**
 * Testes do sistema (sem bibliotecas externas). Cada teste imprime OK ou FALHOU.
 * Uso: java TesteBubbleSort [arquivoEmpresas]
 */
public class TesteBubbleSort {
    private static int ok = 0, falhas = 0;

    public static void main(String[] args) throws Exception {
        System.out.println("=== Testes do Bubble Sort ===");
        testeExemploDoEnunciado();
        testeVetorVazioEUmElemento();
        testeJaOrdenado();
        testeOrdemInversa();
        testeValoresIguaisMantemOrdem();
        testeConversaoCentavos();
        testeAleatorioComparadoComJava();
        testeLeituraEGravacao();
        if (args.length > 0) testeAmostraDoArquivoReal(Path.of(args[0]));
        System.out.printf("%nResultado: %d OK, %d FALHA(S)%n", ok, falhas);
        if (falhas > 0) System.exit(1);
    }

    private static void verificar(String nome, boolean condicao) {
        if (condicao) { ok++; System.out.println("[OK]     " + nome); }
        else { falhas++; System.out.println("[FALHOU] " + nome); }
    }

    private static boolean estaDecrescente(Empresa[] v) {
        for (int i = 0; i + 1 < v.length; i++)
```

```
            if (v[i].getValorCentavos() < v[i + 1].getValorCentavos()) return false;
        return true;
    }

    private static void testeExemploDoEnunciado() {
        Empresa[] v = {
            Empresa.deLinha("Empresa A | 12345678-9 | 5000000"),
            Empresa.deLinha("Empresa B | 98765432-1 | 3000000"),
            Empresa.deLinha("Empresa C | 11223344-5 | 7000000")};
        new BubbleSort().ordenarDecrescente(v);
        verificar("Exemplo do enunciado -> C, A, B",
            v[0].getNomeFantasia().equals("Empresa C") && v[1].getNomeFantasia().equals("Empresa A")
            && v[2].getNomeFantasia().equals("Empresa B"));
    }

    private static void testeVetorVazioEUmElemento() {
        Empresa[] vazio = {};
        new BubbleSort().ordenarDecrescente(vazio);
        Empresa[] um = { Empresa.deLinha("X | 1 | 10.00") };
        new BubbleSort().ordenarDecrescente(um);
        verificar("Vetor vazio e vetor com 1 elemento não quebram", vazio.length == 0 && um.length == 1);
    }

    private static void testeJaOrdenado() {
        Empresa[] v = gerar(1000, 1);
        Arrays.sort(v, Comparator.comparingLong(Empresa::getValorCentavos).reversed());
        BubbleSort b = new BubbleSort();
        b.ordenarDecrescente(v);
        verificar("Vetor já ordenado: 1 passada e 0 trocas (melhor caso O(n))",
            estaDecrescente(v) && b.getPassadas() == 1 && b.getTrocas() == 0);
    }

    private static void testeOrdemInversa() {
        int n = 1000;
        Empresa[] v = new Empresa[n];
        for (int i = 0; i < n; i++) v[i] = new Empresa("E" + i, "0-0", String.valueOf(i));
        BubbleSort b = new BubbleSort();
        b.ordenarDecrescente(v);
        verificar("Ordem inversa (pior caso): n(n-1)/2 trocas = " + (long) n * (n - 1) / 2,
            estaDecrescente(v) && b.getTrocas() == (long) n * (n - 1) / 2);
    }

    private static void testeValoresIguaisMantemOrdem() {
        Empresa[] v = {
            Empresa.deLinha("Primeira | 1 | 100.00"),
            Empresa.deLinha("Maior | 2 | 200.00"),
            Empresa.deLinha("Segunda | 3 | 100.00")};
        new BubbleSort().ordenarDecrescente(v);
        verificar("Valores iguais mantêm a ordem original (algoritmo estável)",
            v[0].getNomeFantasia().equals("Maior") && v[1].getNomeFantasia().equals("Primeira")
            && v[2].getNomeFantasia().equals("Segunda"));
    }

    private static void testeConversaoCentavos() {
        verificar("Conversão de valores para centavos",
            Empresa.converterParaCentavos("652069.19") == 65206919L
            && Empresa.converterParaCentavos("5000000") == 500000000L
            && Empresa.converterParaCentavos("10.5") == 1050L
            && Empresa.converterParaCentavos("0.07") == 7L);
    }

    private static void testeAleatorioComparadoComJava() {
        Empresa[] v = gerar(20_000, 42);
        Empresa[] esperado = v.clone();
        Arrays.sort(esperado, Comparator.comparingLong(Empresa::getValorCentavos).reversed()); // estável
        new BubbleSort().ordenarDecrescente(v);
        verificar("20.000 registros aleatórios: resultado idêntico ao Arrays.sort do Java",
            Arrays.equals(v, esperado));
    }

    private static void testeLeituraEGravacao() throws Exception {
        Path tmp = Files.createTempFile("empresas", ".txt");
        Files.writeString(tmp, "Empresa A | 12345678-9 | 5000000\n\nEmpresa B | 98765432-1 | 3000000\nlinha invalida\n");
        Empresa[] v = ArquivoEmpresas.ler(tmp, 0);
        Path saida = Files.createTempFile("saida", ".txt");
        ArquivoEmpresas.gravar(saida, v);
        String conteudo = Files.readString(saida).replace("\r\n", "\n");
        verificar("Leitura ignora linhas vazias/inválidas e gravação mantém o formato",
            v.length == 2 && conteudo.equals("Empresa A | 12345678-9 | 5000000\nEmpresa B | 98765432-1 | 3000000\n"));
        Files.deleteIfExists(tmp);
        Files.deleteIfExists(saida);
    }

    private static void testeAmostraDoArquivoReal(Path arquivo) throws Exception {
        Empresa[] v = ArquivoEmpresas.ler(arquivo, 50_000);
        Empresa[] esperado = v.clone();
        Arrays.sort(esperado, Comparator.comparingLong(Empresa::getValorCentavos).reversed());
        long t = System.nanoTime();
        new BubbleSort().ordenarDecrescente(v);
        double s = (System.nanoTime() - t) / 1e9;
        verificar(String.format("Primeiros 50.000 registros do arquivo real ordenados corretamente (%.2f s)", s),
            estaDecrescente(v) && Arrays.equals(v, esperado));
    }

    private static Empresa[] gerar(int n, long semente) {
```

```
        Random r = new Random(semente);
        Empresa[] v = new Empresa[n];
        for (int i = 0; i < n; i++) {
            long centavos = r.nextInt(1_000_000_000);
            v[i] = new Empresa("Empresa " + i, String.format("%08d-%d", r.nextInt(100_000_000), r.nextInt(10)),
                    (centavos / 100) + "." + String.format("%02d", centavos % 100));
        }
        return v;
    }
}
```